

# Intel HEX Bestandsformaat & Assembly Mapping

---

Dit document legt uit hoe het **Intel HEX** bestandsformaat ( `.hex` ) is opgebouwd, hoe machinecode zich verhoudt tot assembly-instructies en hoe je een HEX-bestand leest. Dit is een fundamenteel onderdeel van computerarchitectuurlessen.

---

## 1. Waarom gebruiken we Intel HEX?

---

Wanneer je C-code compileert, genereert de compiler uiteindelijk binaire machinecode (nullen en enen) die in het flashgeheugen van de microcontroller moet worden geladen. In plaats van een rauw binair bestand ( `.bin` ), gebruiken we in de embedded wereld vaak het **Intel HEX** formaat ( `.hex` ).

- Het is een **tekstbestand (ASCII)**, waardoor het menselijk leesbaar is en makkelijk te bewerken met een tekst-editor.
  - Elke regel bevat een **checksum** om te controleren of er transmissiefouten zijn opgetreden tijdens het uploaden.
  - Elke regel specificeert expliciet het **geheugenadres** waar de data geschreven moet worden, wat handig is als het geheugen niet-contiguous (gefragmenteerd) is.
- 

## 2. De Anatomie van een HEX-Record (Regel)

---

Elke regel in een Intel HEX-bestand wordt een **record** genoemd. Een record begint altijd met een dubbele punt `:` en heeft de volgende vaste structuur:

```
: [Byte Count] [Address] [Record Type] [Data Bytes ...] [Checksum]
```

Laten we de velden ontleden:

1. **Start Code ( `:` )**: 1 ASCII-karakter. Geeft het begin van een record aan.
2. **Byte Count (1 byte / 2 hex karakters)**: Het aantal data-bytes dat in dit record zit (dus exclusief adres, type en checksum). Meestal `10` (16 bytes) of kleiner.
3. **Address (2 bytes / 4 hex karakters)**: Het 16-bit startadres in het geheugen waar de data van dit record moet worden geplaatst.
4. **Record Type (1 byte / 2 hex karakters)**: Bepaalt de betekenis van het record:
  - `00` = **Data Record**: Bevat daadwerkelijke machinecode of programmadatabytes.
  - `01` = **End of File (EOF) Record**: Geeft het einde van het bestand aan. Heeft altijd byte count `00` en adres `0000` (bijv. `:000000001FF` ).

- **02** = **Extended Segment Address**: Voor 8086-achtige segmentatie.
  - **04** = **Extended Linear Address**: Wordt gebruikt om 32-bit adressen aan te geven (voor grotere flash-geheugens dan 64KB).
5. **Data**: De eigenlijke bytes van het programma (lengte is gelijk aan *Byte Count*).
  6. **Checksum (1 byte / 2 hex karakters)**: Een wiskundige controlewaarde. Het is het **two's complement** van de som van alle voorgaande bytes op die regel (exclusief de `:`).

## Hoe bereken je de Checksum?

1. Tel alle bytes op de regel bij elkaar op (Byte Count + Address High + Address Low + Type + alle Data bytes).
2. Neem de minst significante byte (som modulo 256).
3. Trek dit af van `0x100` (dit is het two's complement).
4. Het resultaat is de checksum byte. Als je alle bytes inclusief de checksum optelt, moet het resultaat modulo 256 altijd `00` zijn.

## 3. Van Assembly naar HEX: Praktische Voorbeelden

Hieronder staan twee concrete voorbeelden die laten zien hoe C-code wordt vertaald naar Assembly-opcodes (machinecode) en hoe dit in de HEX-file terechtkomt.

### Voorbeeld A: De Reset Vector (Sprong naar de code)

Wanneer de CH552 microcontroller opstart, begint de program counter (PC) op adres `0x0000`. Op dit adres staat meestal een sprong-instructie naar het echte startpunt van het programma (omdat interrupts de opvolgende adressen bezetten).

- **Assembly instructie**: `LJMP 0x0040` (Long Jump naar adres `0x0040`)
- **Machinecode (Hex)**:
  - Opcode voor `LJMP` = `0x02`
  - Doeladres = `0x0040` (Hoog byte: `0x00`, Laag byte: `0x40`)
  - De resulterende bytes in het geheugen zijn: `02 00 40`

Als deze instructie direct op adres `0x0000` staat, ziet het record in de HEX-file er als volgt uit:

```
:03000000020040BB
```

### Ontleding:

- `:` : Start code
- `03` : Er zijn 3 bytes data (`02`, `00`, `40`).
- `0000` : Schrijf deze bytes naar adres `0x0000`.
- `00` : Record type `00` (Data Record).

- `020040` : De machinecode voor `LJMP 0x0040`.
- `BB` : Checksum.
  - *Berekening:*  $03 + 00 + 00 + 00 + 02 + 00 + 40 = 0x45$ .
  - Two's complement:  $0x100 - 0x45 = 0xBB$ .

## Voorbeeld B: Een Bit-adresseerbare I/O Pin aanzetten

In onze blinker-code schrijven we `P3_0 = 1;` om de LED aan te zetten.

- **Assembly instructie:** `SETB 0xB0` (Set Bit op adres `0xB0`. Adres `0xB0` is bit 0 van Port 3).
- **Machinecode (Hex):**
  - Opcode voor `SETB bit` = `0xD2`
  - Bit-adres van P3.0 = `0xB0`
  - De resulterende bytes in het geheugen zijn: `D2 B0`

Als deze instructie in het programma op adres `0x0040` staat, ziet het HEX-record er zo uit:

```
:02004000D2B03C
```

### Ontleding:

- `:` : Start code
- `02` : Er zijn 2 bytes data (`D2`, `B0`).
- `0040` : Schrijf deze bytes naar adres `0x0040`.
- `00` : Record type `00` (Data Record).
- `D2B0` : De machinecode voor `SETB 0xB0`.
- `3C` : Checksum.
  - *Berekening:*  $02 + 00 + 40 + 00 + D2 + B0 = 0x1C4$ .
  - Modulo 256:  $0x1C4 \pmod{256} = 0xC4$ .
  - Two's complement:  $0x100 - 0xC4 = 0x3C$ .

## 4. Een compleet mini HEX-bestand lezen

Hier is een voorbeeld van een compleet (fictief) Intel HEX-bestand voor een heel klein programma:

```
:03000000020040BB
:02004000D2B03C
:02004200C2B04C
:0200440080FA3E
:00000001FF
```

## Betekenis van dit programma:

1. **Regel 1 ( :03000000020040BB )**: Op adres `0x0000` (reset vector) springen we direct naar de hoofdcode op `0x0040` ( `LJMP 0x0040` ).
2. **Regel 2 ( :02004000D2B03C )**: Op adres `0x0040` zetten we poort P3.0 hoog ( `SETB P3.0` ).
3. **Regel 3 ( :02004200C2B04C )**: Op adres `0x0042` zetten we poort P3.0 laag ( `CLR P3.0` . Opcode voor `CLR` bit is `0xC2` ).
4. **Regel 4 ( :0200440080FA3E )**: Op adres `0x0044` springen we terug naar adres `0x0040` om een oneindige lus te creëren ( `SJMP 0x0040` . Opcode for `SJMP` offset is `0x80` , en `0xFA` is de relative jump offset in two's complement, wat terugspringt naar `0x0040` ).
5. **Regel 5 ( :000000001FF )**: Einde van het bestand (End of File).

---

## Didactische tip voor de les:

Laat studenten met een hex-editor of simpelweg in VS Code hun eigen gecompileerde `build/ch552_blink.hex` openen en vergelijken met de gegenereerde assembly in `build/main.asm` (die door SDCC wordt gemaakt). Laat hen de machine-bytes ( `02` , `D2` , `C2` ) opzoeken in de HEX-file om te bewijzen dat wat ze in C schrijven, daadwerkelijk als deze specifieke byte-patronen in het flash-geheugen belandt!